

Interrupt-Driven Real-Time Temperature Monitoring System Using STM32-Nucelo

ENGR 478 – Design with Microprocessors

Team Chevron

Avery Whann – 918781869, awhann@sfsu.edu

Spring 2026

Professor Xiaorong Zhang



Table of Contents

1.	Introduction	2
1.1	Background.....	2
1.2	Motivation.....	2
1.3	Objectives.....	3
2.	Design and Implementation	4
2.1	Hardware Architecture	4
2.2	Software Architecture	8
2.3	Temperature Sensing Module	9
2.4	ADC Module.....	10
2.5	UART Communication Module.....	11
2.6	Push-Button Interrupt Module.....	11
2.7	Software Flowchart.....	12
3.	Procedure.....	13
3.1	ADC and Temperature Response Test.....	13
3.2	UART Communication Test	18
3.3	Push-Button Interrupt and SysTick Debounce Test.....	20
4.	Results and Discussion.....	23
4.1	ADC and Temperature Response Results.....	23
4.2	UART Communication Results	24
4.3	Push-Button Interrupt and Debounce Results.....	25
4.4	Design Constraints	26
4.5	Economic Constraint	26
4.6	Sustainability Constraint.....	27
4.7	Environmental Constraint	28
5.	Roles of Team Members	28
6.	Conclusion and Future Work	29
7.	References	30
8.	AI Use Statement.....	31

1. Introduction

1.1 Background

Embedded temperature monitoring systems are widely used in industrial control systems, environmental monitoring, consumer electronics, and portable sensing devices. Modern microcontrollers integrate peripherals such as analog-to-digital converters (ADCs), timers, external interrupts, and serial communication interfaces, allowing compact sensing systems to be implemented using minimal external hardware.

Analog temperature sensors provide a simple and cost-effective method for measuring environmental temperature. The TMP36 temperature sensor produces an analog output voltage linearly proportional to temperature, allowing direct interfacing with microcontroller ADC peripherals for digital processing and analysis (Analog Devices, n.d.).

Low-power and event-driven system design are important considerations in portable embedded applications. Instead of continuously sampling sensor data, interrupt-driven architectures allow measurements to occur only when requested by the user, reducing unnecessary peripheral activity and processor utilization, thereby reducing overall battery power consumption.

This project uses the STM32L476RG microcontroller and a TMP36 analog temperature sensor to implement an interrupt-driven temperature monitoring system capable of ADC acquisition, temperature conversion, and UART serial communication. The system was designed with future battery-powered operation in mind, where minimizing unnecessary processing and peripheral activity is important for extending portable device battery life.

1.2 Motivation

The motivation for this project was to gain practical experience with event-driven embedded system design in applications where energy conservation is an important consideration. Instead of continuously polling the temperature sensor, the system was designed to perform measurements only when requested through a push-button interrupt event.

This approach provided experience with external interrupt handling, SysTick-based debounce timing, ADC acquisition, and UART communication while exploring how interrupt-driven architectures can reduce unnecessary processor and peripheral activity in portable embedded systems.

The project also provided an opportunity to become more familiar with low-power design concepts commonly used in battery-powered sensing and monitoring applications.

1.3 Objectives

The objectives of this project were to:

- Interface the TMP36 temperature sensor with the STM32 ADC.
- Convert analog voltage measurements into temperature values.
- Implement interrupt-driven temperature sampling using a push-button input.
- Develop an event-driven temperature measurement system using SysTick-based debounce logic.
- Transmit sensor data through UART serial communication to a virtual COM port.
- Assess the economic feasibility of the system by utilizing low-cost and readily available components.
- Consider sustainability constraints through efficient power usage and minimal hardware requirements.
- Evaluate environmental constraints by examining the system's operation under different ambient temperature conditions and promoting reduced electronic waste through reusable development hardware.

2. Design and Implementation

2.1 Hardware Architecture

The hardware architecture of the system consists of a TMP36 analog temperature sensor, a push-button interrupt input, and the STM32L476RG microcontroller. The TMP36 sensor output is connected to the ADC input pin (PA0), while the push-button is connected to the external interrupt pin (PC2). Temperature data is processed by the microcontroller and transmitted through USART2 to a virtual COM port. **Figure 1** illustrates the overall system architecture.

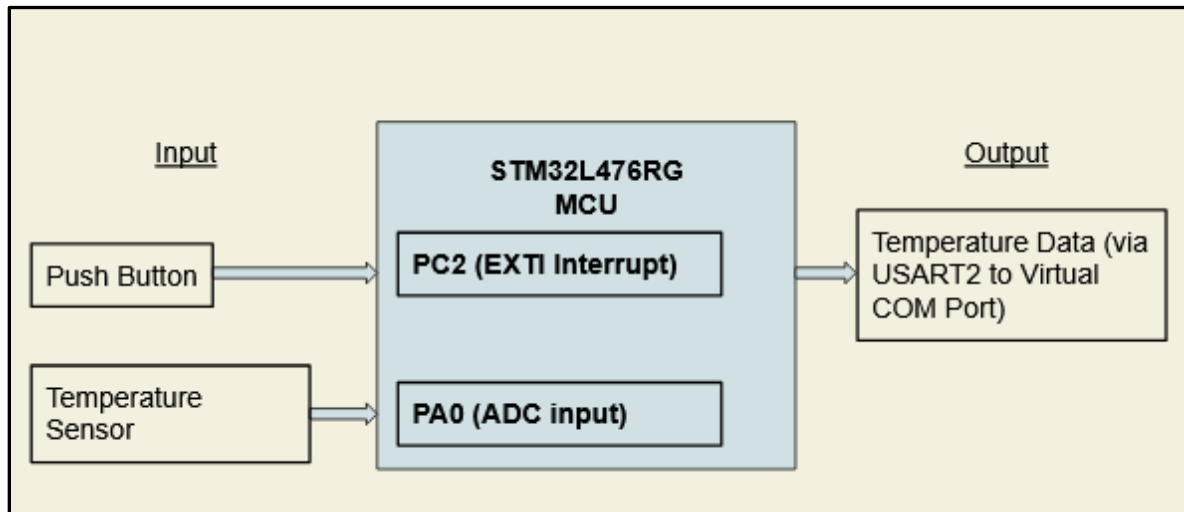


Figure 1: Overall system architecture of the interrupt-driven STM32 temperature monitoring system.

The physical implementation of the system was constructed using the STM32L476RG Nucleo development board, a TMP36 analog temperature sensor, and a push-button input connected through a breadboard interface. The TMP36 sensor output was connected to pin PA0 for ADC acquisition using ADC1 channel 5.

The push button was connected to pin PC2 and configured using the STM32 internal pull-up resistor for EXTI interrupt detection. The internal pull-up resistance, typically 40 k Ω , maintained the GPIO input at a stable logic HIGH state while the button remained unpressed, preventing floating input behavior and reducing susceptibility to electrical noise (STMicroelectronics, 2020). When the push button was pressed, PC2 was driven directly to ground, generating a falling-edge transition used to trigger the external interrupt event.

Using the internal pull-up resistor also eliminated the need for an additional external resistor component while simplifying the hardware design. Applying Ohm's Law demonstrates that the pull-up configuration results in minimal current consumption when the button is pressed:

$$I = \frac{V}{R} = \frac{3.3 \text{ V}}{40 \text{ k}\Omega} = 82.5 \mu\text{A}$$

This small current draw maintained a stable digital input level while supporting low-power embedded system operation.

USART2 communication was routed through the onboard virtual COM port interface for external serial data transmission and logging.

Figure 2 shows the physical hardware implementation of the system, while **Figure 3** illustrates the circuit connections between the STM32L476RG microcontroller, TMP36 temperature sensor, and push-button interrupt input.

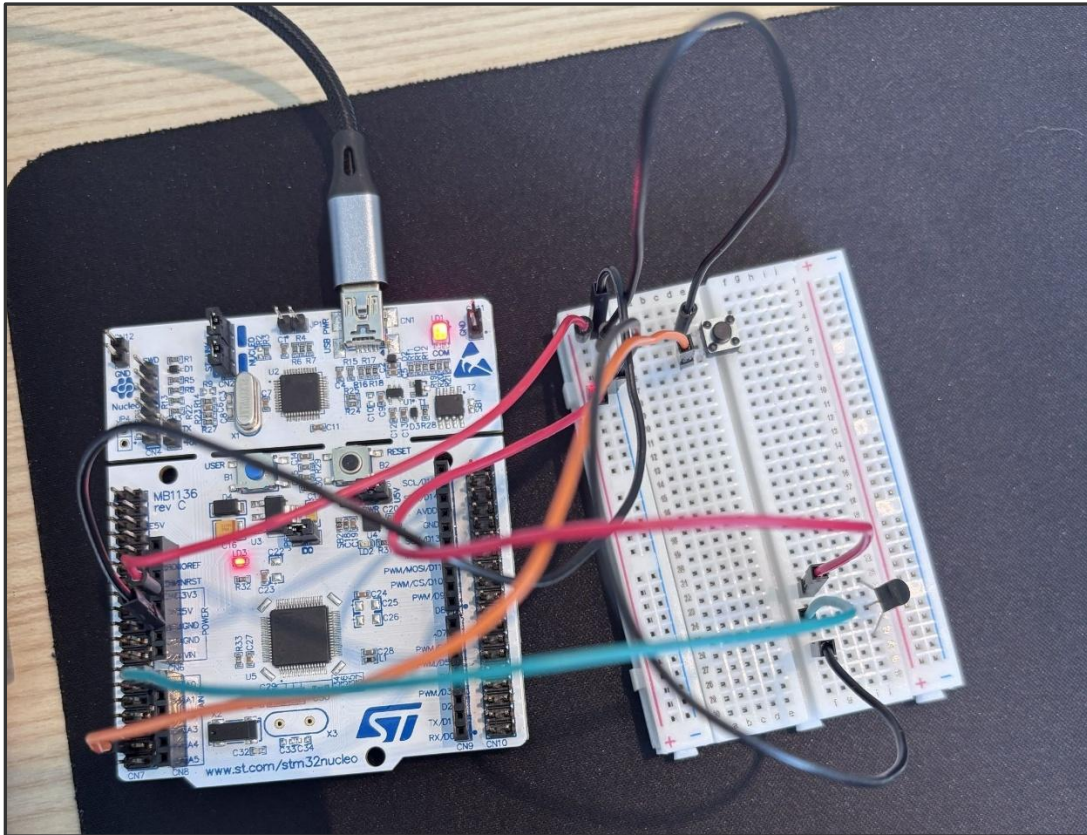


Figure 2: Physical implementation of the STM32 TMP36 temperature monitoring system.

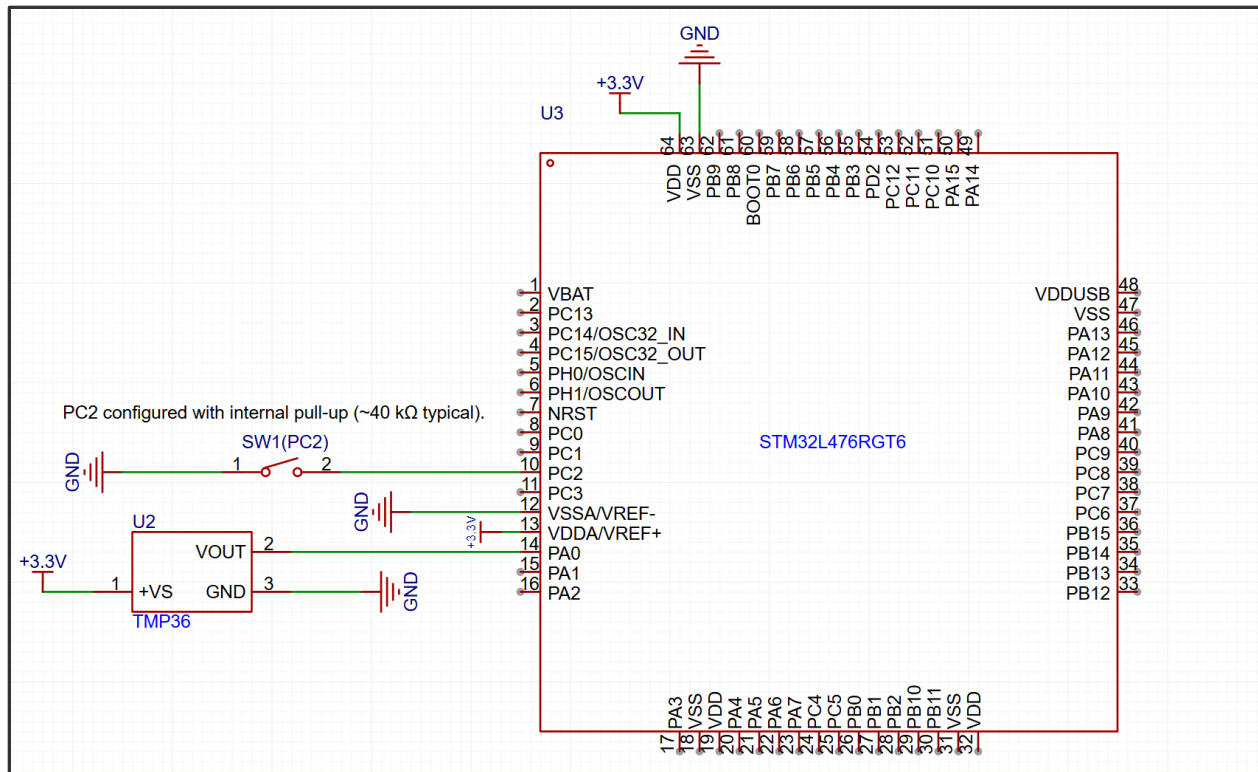


Figure 3: Simplified schematic of the STM32L476-based temperature sensing and push-button input circuit.

The schematic shows the primary functional connections of the system. The circuit uses an STM32L476RG Nucleo board interfaced with a TMP36 temperature sensor and a push button for digital input. The TMP36 sensor is powered using the board's 3.3 V supply, with its output connected to pin PA0, which is configured as an analog input for the STM32 ADC. The sensor produces an analog voltage proportional to temperature, allowing the microcontroller to measure and process temperature values.

A push button is also connected to the system for user input. One terminal of the push button is connected to pin PC2, while the other terminal is connected to ground (GND). The GPIO pin is configured using the STM32's internal 40 kΩ pull-up resistor, ensuring that the pin remains at a logic HIGH level when the button is not pressed. When the button is pressed, the pin is pulled to ground, generating a logic LOW signal that can be detected by the microcontroller and used to trigger an external interrupt event.

2.2 Software Architecture

The software architecture of the system was designed using a modular embedded C structure. Separate source and header files were used to isolate system functionality into independent modules.

The primary software modules included:

- **main.c**
- **temperature.c / temperature.h**
- **systick_timer.c / systick_timer.h**
- **button.c / button.h**
- **usart.c / usart.h**

The corresponding header files contained function prototypes, shared macros, and module declarations used throughout the software architecture.

The **main.c** module handled overall program initialization and system control flow. During startup, it initialized SysTick timing, push-button interrupt configuration, ADC setup, and USART2 communication. The main loop used the **temp_request** software flag to determine when a button-triggered temperature measurement was requested. When the flag was set, the program collected 64 ADC samples and averaged the result to reduce analog measurement noise. The averaged ADC value was then converted into millivolts and processed using fixed-point integer arithmetic to calculate the TMP36 temperature in degrees Celsius. Finally, CSV-formatted ADC, voltage, and temperature data were transmitted through the USART2 interface for external monitoring and MATLAB-based data visualization.

The **temperature.c** module handled ADC configuration for the TMP36 sensor interface. This module configured PA0 as an analog input, connected the analog switch, enabled the ADC peripheral clock, and initialized ADC1 for sensor voltage acquisition.

The **Systick_timer.c** module provided millisecond timing functions for button debounce handling and timing control. The module utilized the STM32 SysTick peripheral to generate software timing delays during push-button operation.

The **button.c** module implemented external interrupt handling for push-button input detection on PC2. The push-button input was configured with an internal pull-up resistor and negative-logic operation. EXTI2 was configured to trigger on the falling edge only, while the rising-edge trigger was disabled to prevent release events from generating additional interrupts.

The **usart.c** module handled UART serial communication for external data transmission. Temperature data was transmitted through USART2 to a virtual COM port.

2.3 Temperature Sensing Module

The temperature sensing module was implemented using the TMP36 analog temperature sensor. The TMP36 produces an analog output voltage proportional to the measured ambient temperature.

The TMP36 sensor provides an output voltage of 500 mV at 0°C with a scale factor of 10 mV/°C (Analog Devices, n.d.). Temperature values were calculated from the measured sensor voltage using the following transfer function, while the corresponding TMP36 output characteristics are summarized below in **Table 1**:

$$T\text{ }^{\circ}\text{C} = \frac{V_{mV} - 500}{10}$$

Table 1: TMP36 output characteristics (Analog Devices, n.d.).

Sensor	Offset Voltage (V)	Output Voltage Scaling (mV/°C)	Output Voltage at 25°C (mV)
TMP35	0	10	250
TMP36	0.5	10	750
TMP37	0	20	500

The TMP36 sensor was selected for this project because it provides a 500 mV offset at 0°C, requires no external calibration, and produces a linear output scaling factor of 10 mV/°C, simplifying software-based temperature conversion (Analog Devices, n.d.).

2.4 ADC Module

The ADC module was used to convert the analog output voltage of the TMP36 temperature sensor into digital values for software processing. The TMP36 output was connected to PA0, which was configured as an analog input and connected to the STM32 analog switch. On the STM32L476RG, PA0 corresponds to ADC1 channel 5 (STMicroelectronics, 512).

ADC1 was configured in independent, single-ended conversion mode with 12-bit resolution and right-aligned output data. Before operation, the ADC was enabled through the peripheral clock, removed from deep-power-down mode, and calibrated before operation. The ADC regular sequence was configured for one conversion on channel 5, and the sample time was increased to improve measurement stability.

The 12-bit ADC produced digital values in the range of 0 to 4095 (STMicroelectronics, 2020). Measured ADC values were converted into millivolt readings using the following relationship:

$$V_{mV} = \frac{ADC_{value} \times 3300 \text{ (3.3 V)}}{4095}$$

To reduce noise, multiple ADC samples were collected and averaged before voltage and temperature calculations were performed. The averaged voltage value was then passed to the temperature conversion logic to calculate the measured temperature.

2.5 UART Communication Module

The UART communication module was used to transmit temperature measurement data from the STM32L476RG microcontroller to an external PC terminal for monitoring and analysis.

USART2 was configured as the primary serial communication interface and connected to the virtual COM port provided through the Nucleo development board USB interface (STMicroelectronics, 2015). USART2 was configured for asynchronous serial communication at a baud rate of 115,200 bps, using the STM32 virtual COM port interface.

The UART subsystem also transmitted formatted ADC, voltage, and temperature data after each button-triggered measurement. Serial output data was displayed and logged using the PuTTY terminal application.

2.6 Push-Button Interrupt Module

The push-button interrupt module was implemented to allow user-triggered temperature measurements. The push button was connected to pin PC2 and configured as a GPIO input with an internal pull-up resistor. The PC2 input was mapped to EXTI2, allowing the system to detect a falling-edge signal when the button was pressed.

When a valid button press occurred, the EXTI2 interrupt handler cleared the pending interrupt flag and used the SysTick timer to apply a 50 ms software debounce delay.

After debounce validation, the interrupt service routine incremented a button press counter made for debugging, and set the temp_request flag. The main loop used this flag to determine when to perform ADC sampling, voltage conversion, temperature calculation, and UART transmission. This flag-based design kept the interrupt service routine short and separated interrupt handling from the main processing logic.

2.7 Software Flowchart

The software operation of the system was designed using an interrupt-driven architecture. The main program continuously waits for a temperature request flag generated by the push-button external interrupt (EXTI) routine. When a valid button press is detected, the interrupt routine sets the temp_request flag, allowing the main loop to perform ADC sampling, voltage conversion, temperature calculation, and UART data transmission. **Figure 4** illustrates the overall software flow of the system.

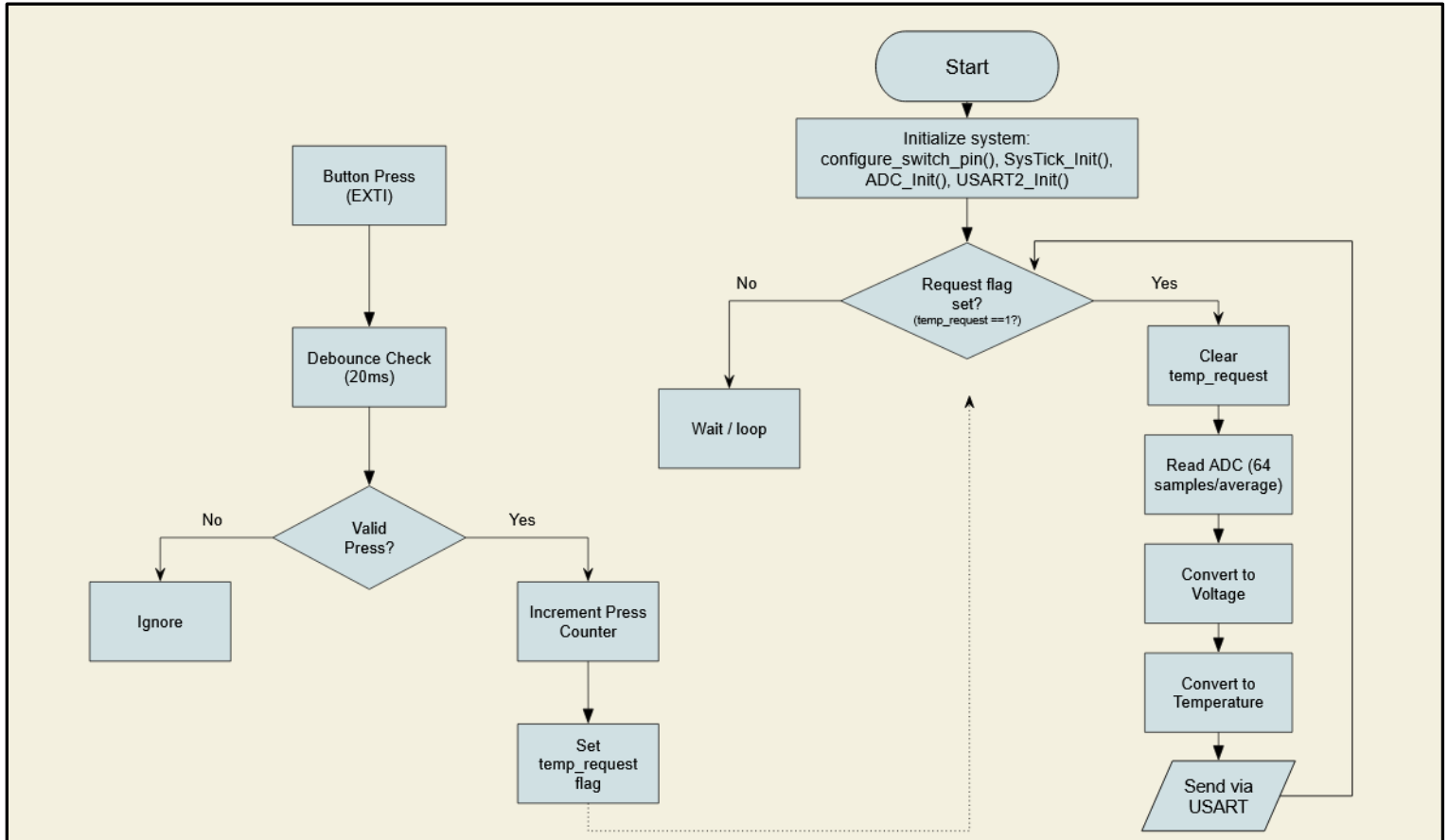


Figure 4: Software flowchart for interrupt-driven temperature acquisition and UART transmission.

3. Procedure

3.1 ADC and Temperature Response Test

Test Procedure:

1. PuTTY was configured to monitor UART temperature sampling data through the STM32 virtual COM port interface.
2. The TMP36 temperature sensor was initially placed in an ice bath to expose the sensor to a low-temperature environment.
3. ADC values, voltage measurements, and calculated temperature data were recorded while the sensor stabilized in the ice bath.
4. The TMP36 sensor was removed from the ice bath and exposed to room-temperature ambient conditions.
5. UART output data was monitored and recorded while the sensor temperature gradually returned to room temperature.
6. The TMP36 sensor was then exposed to warm water to increase the sensor temperature above ambient conditions.
7. ADC values, voltage measurements, and calculated temperature data were continuously monitored during sensor warming.
8. UART output data was recorded in CSV format for subsequent temperature trend visualization and analysis using MATLAB.

Measurement Method:

- Signal measured: TMP36 analog output voltage converted through the STM32 ADC.
- Tools used: STM32 ADC subsystem, PuTTY terminal application, and MATLAB data visualization.
- Measurement method: UART monitoring and temperature trend analysis under varying temperature conditions.
- Verification objective: Confirm ADC response and temperature conversion behavior across low, ambient, and elevated temperature conditions.

Instrument Configuration:

- Sensor device: TMP36 temperature sensor.
- ADC input source: TMP36 analog output voltage.
- ADC resolution: 12-bit ADC conversion.
- UART monitoring tool: PuTTY terminal application.
- Serial interface configuration: COM3 virtual COM port at 115,200 bps.
- Data output format: CSV-formatted ADC, voltage, and temperature data.

Figure 5 shows the MATLAB environment and plotting script used to import CSV-formatted UART temperature data and generate the TMP36 temperature response visualization. The MATLAB script imported the recorded UART data, extracted the temperature measurements, and plotted temperature as a function of sample number for thermal response analysis.

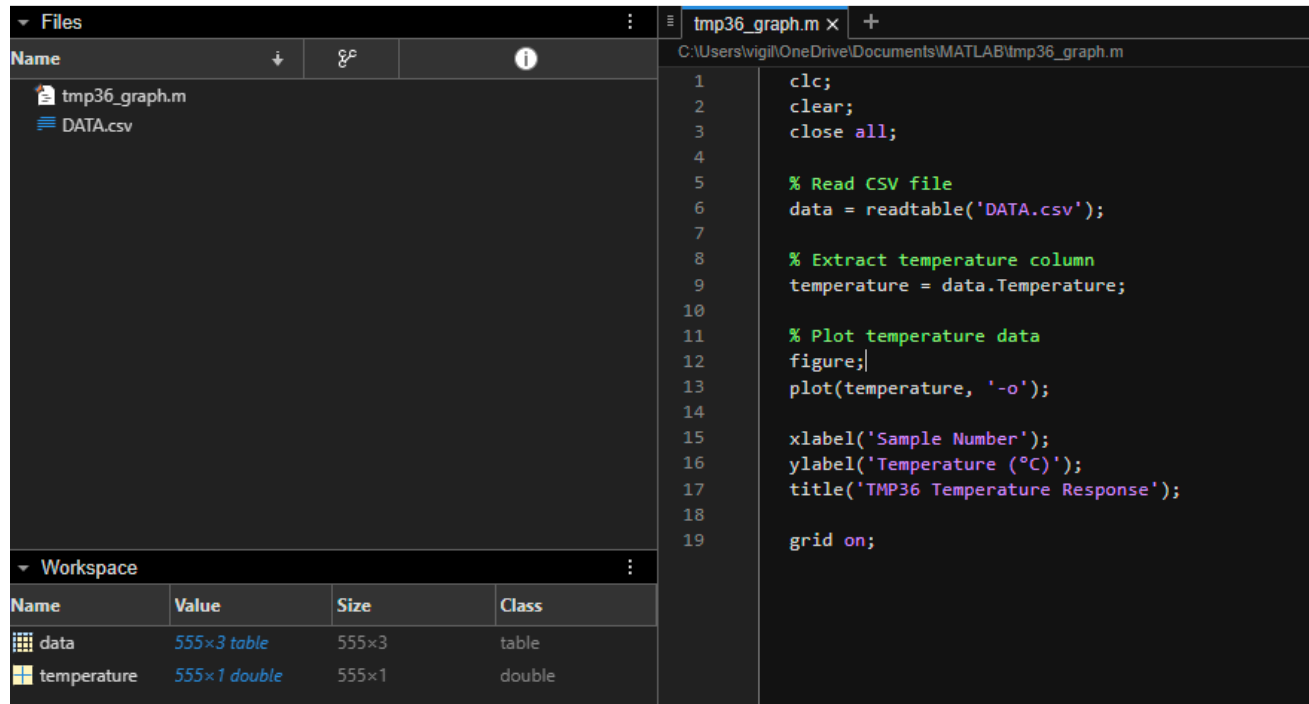


Figure 5: MATLAB script used for CSV-based TMP36 temperature response visualization and analysis.

Figure 6 shows the TMP36 temperature sensor during low-temperature testing using an ice bath environment. The sensor was exposed to reduced temperatures to verify ADC response and temperature conversion behavior under cold operating conditions.

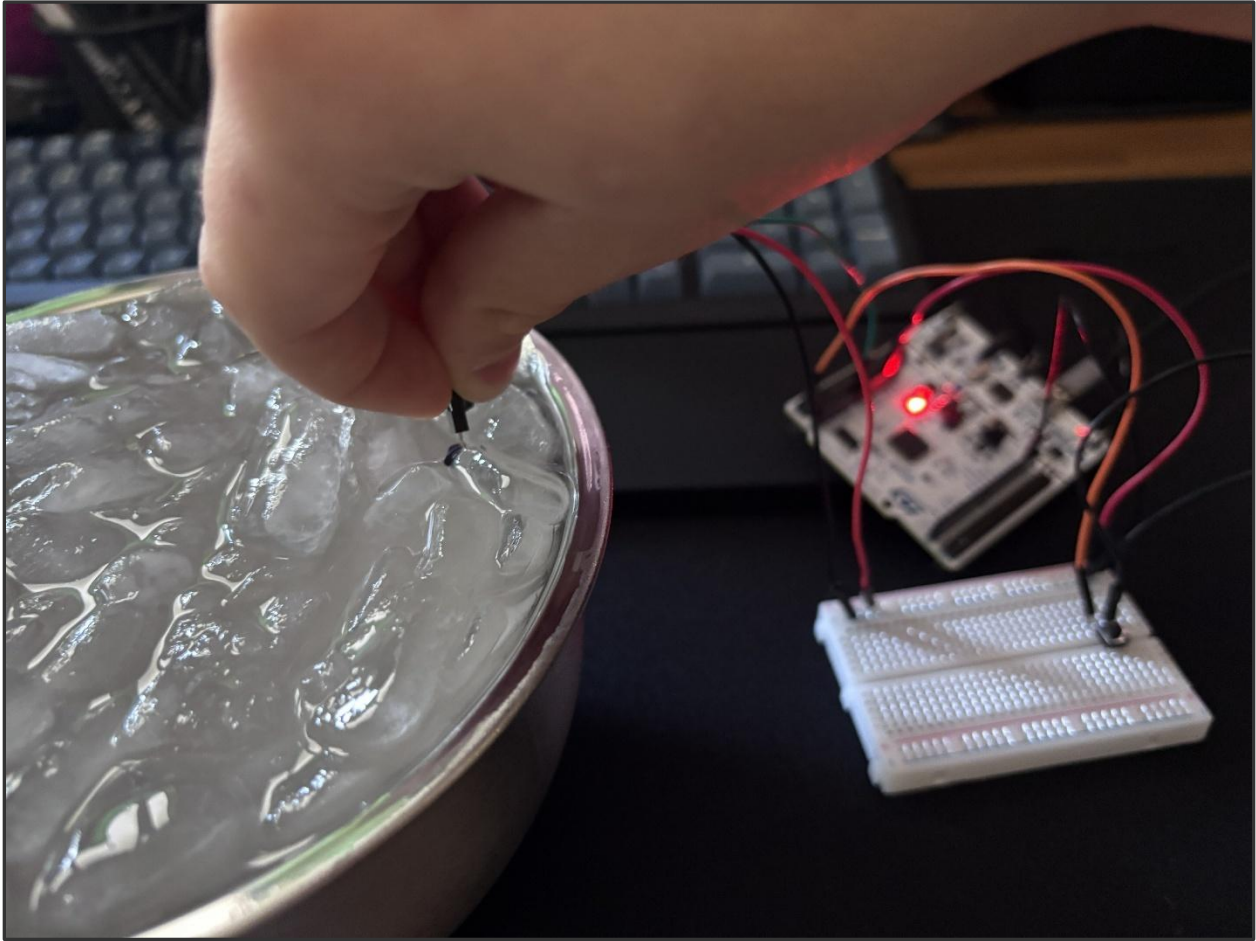


Figure 6: TMP36 sensor during ice-bath temperature response testing.

Figure 7 shows the TMP36 sensor during ambient temperature recovery after removal from the ice bath. UART temperature data was monitored while the sensor gradually returned to room-temperature conditions, allowing observation of ADC stabilization and temperature conversion behavior during thermal recovery.

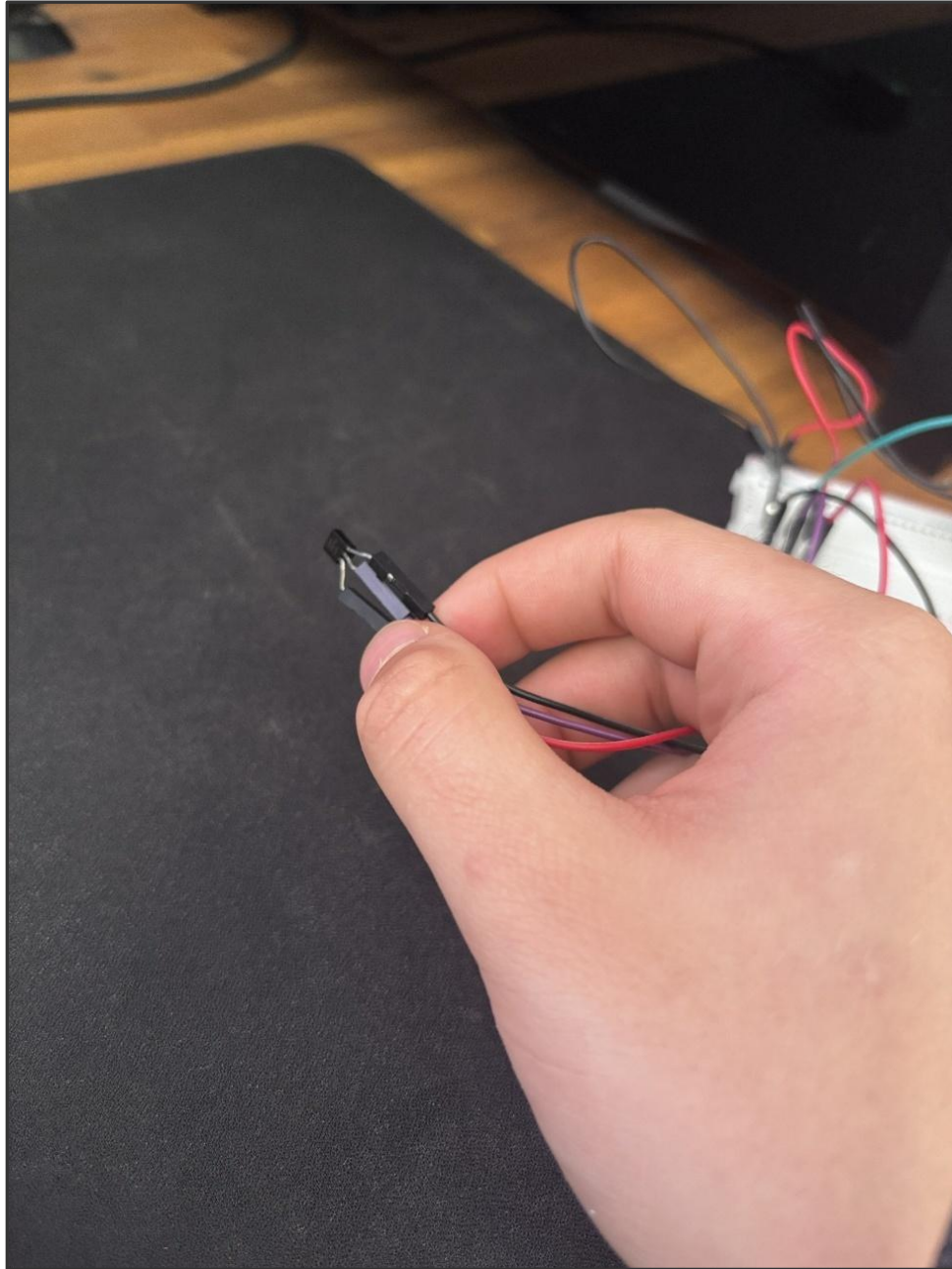


Figure 7: TMP36 sensor during room-temperature recovery following ice-bath exposure.

Figure 8 shows the TMP36 sensor during elevated-temperature testing using warm-water exposure. The sensor was exposed to a higher-temperature environment to verify ADC response and temperature conversion behavior under elevated thermal conditions.

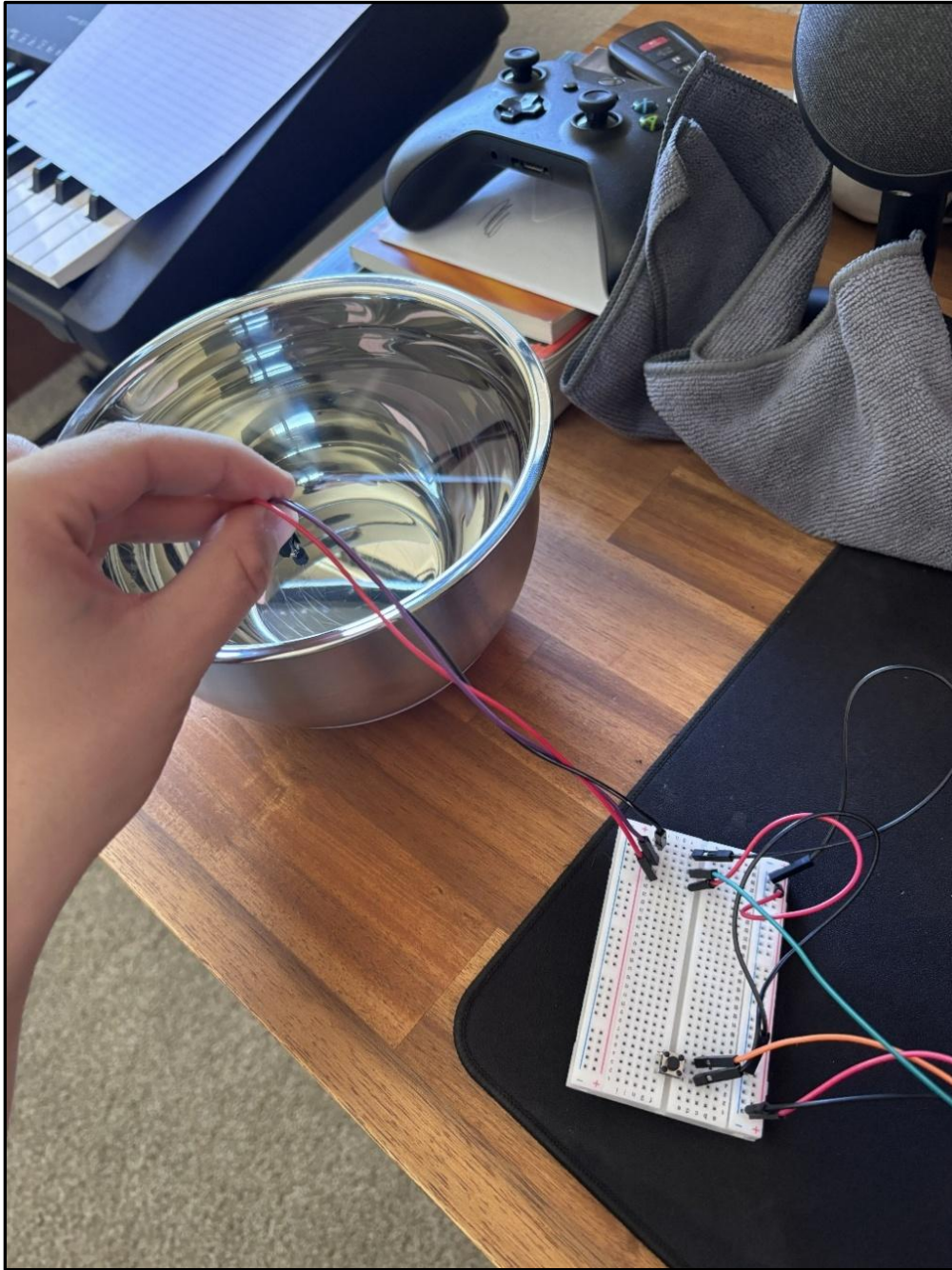


Figure 8: TMP36 sensor during elevated-temperature testing using warm-water exposure.

3.2 UART Communication Test

Test Procedure:

1. PuTTY was configured for serial communication using the COM port assigned to the STM32 Nucleo board.
2. UART communication parameters were configured.
3. The TMP36 temperature sensing system was initialized and powered.
4. The user push-button was pressed to trigger temperature sampling and UART data transmission.
5. ADC values, voltage measurements, and temperature data were monitored using the PuTTY terminal application.
6. Multiple button-triggered measurements were performed to verify reliable serial communication and consistent CSV-formatted data output.
7. UART output data was recorded for subsequent temperature trend visualization and analysis.

Measurement Method:

- Signal measured: UART serial output data.
- Tool used: PuTTY terminal application.
- Measurement method: Serial monitoring and observation of transmitted ADC, voltage, and temperature data.

Instrument Configuration:

- Communication interface: USART2 virtual COM port
- Baud rate: 115,200 bps
- Data bits: 8
- Stop bits: 1
- Parity: None
- Output format: CSV-formatted serial data

Figure 9 shows the PuTTY serial communication configuration used during UART monitoring of the STM32 USART2 virtual COM interface operating at 115,200 bps.

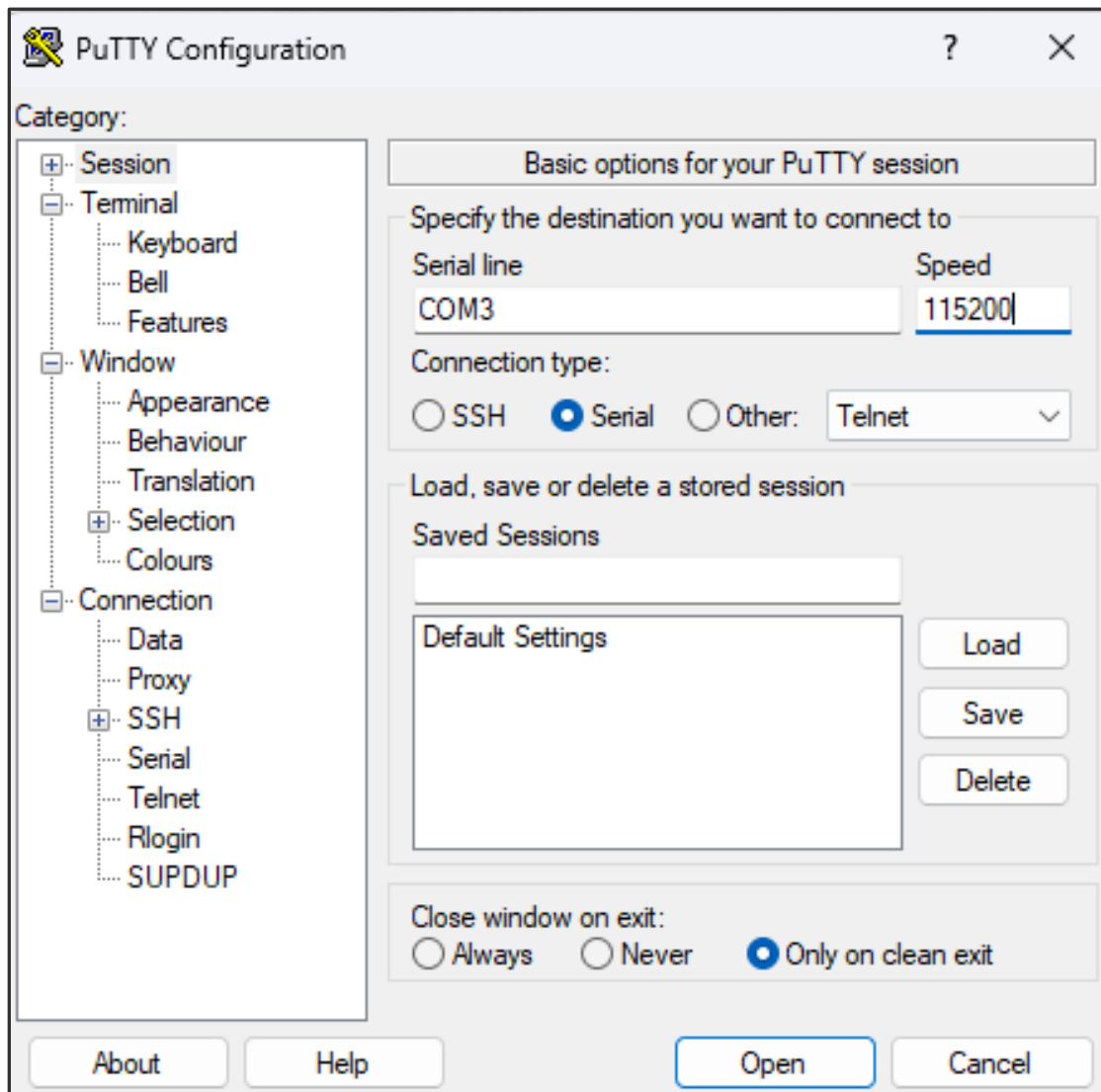


Figure 9: PuTTY serial communication configuration for USART2 monitoring at 115200 bps.

3.3 Push-Button Interrupt and SysTick Debounce Test

Test Procedure:

1. The user push-button was configured as a GPIO external interrupt input with an internal pull-up resistor enabled.
2. The external interrupt was configured to trigger on the falling-edge transition generated by a push-button press.
3. An Analog Discovery 2 (AD2) logic analyzer was connected to the PC2 push-button signal line for waveform observation.
4. The push-button was pressed repeatedly while monitoring the digital waveform using the Waveforms logic analyzer software.
5. Falling-edge transitions and push-button signal behavior generated during button actuation were observed on the raw push-button signal.
6. ADC conversion and UART temperature transmission behavior were monitored using the PuTTY terminal application after each interrupt event.
7. Multiple normal, quick, and light button presses were performed to verify stable debounce operation and confirm that only one temperature sampling event occurred for each valid button press.

Measurement Method:

- Signal measured: Raw Push-button signal and UART temperature sampling events.
- Tool used: Analog Discovery 2 (AD2) logic analyzer with WaveForms software and PuTTY terminal application.
- Measurement method: Digital waveform observation and UART output monitoring.
- Verification objective: Confirm falling-edge interrupt triggering, negative-logic push-button operation, and stable SysTick software debounce behavior.

Instrument Configuration:

- Verification instruments: Analog Discovery 2 (AD2) logic analyzer with Waveforms software.
- Time scale: 500 ms/div.
- Sampling rate: 800 Hz.
- Logic analyzer channel mapping: PC2 → AD2 digital input (DIO0).
- Trigger configuration: Falling-edge digital trigger.
- Serial interface configuration: COM3 virtual COM port at 115,200 bps.

Figure 10 shows the Waveforms logic analyzer configuration and timing measurements used to monitor the PC2 push-button interrupt signal during testing. The Analog Discovery 2 (AD2) digital input channel was connected to the PC2 signal line to observe falling-edge transitions, negative-logic operation, and push-button signal behavior during interrupt generation and software debounce verification.

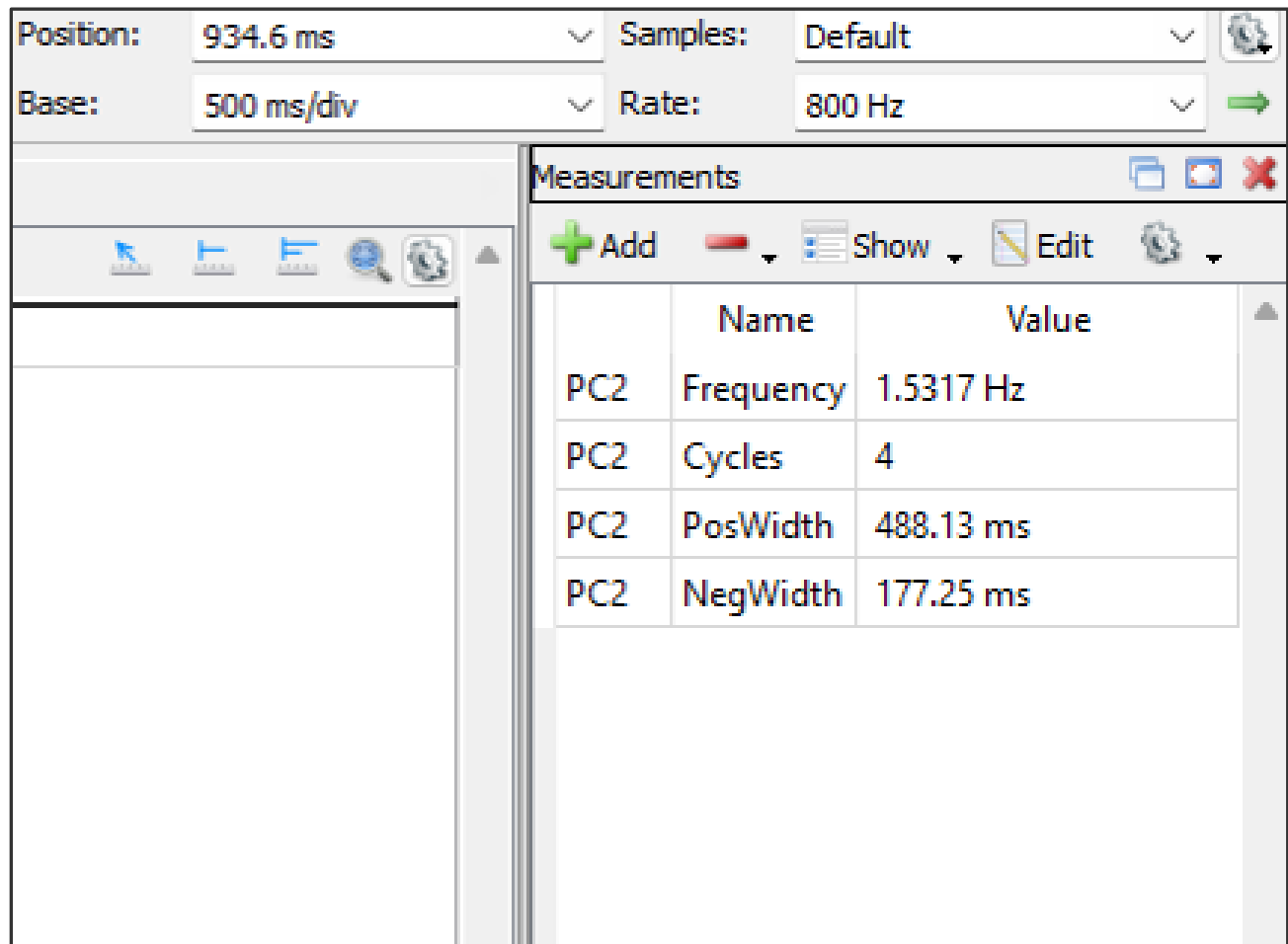
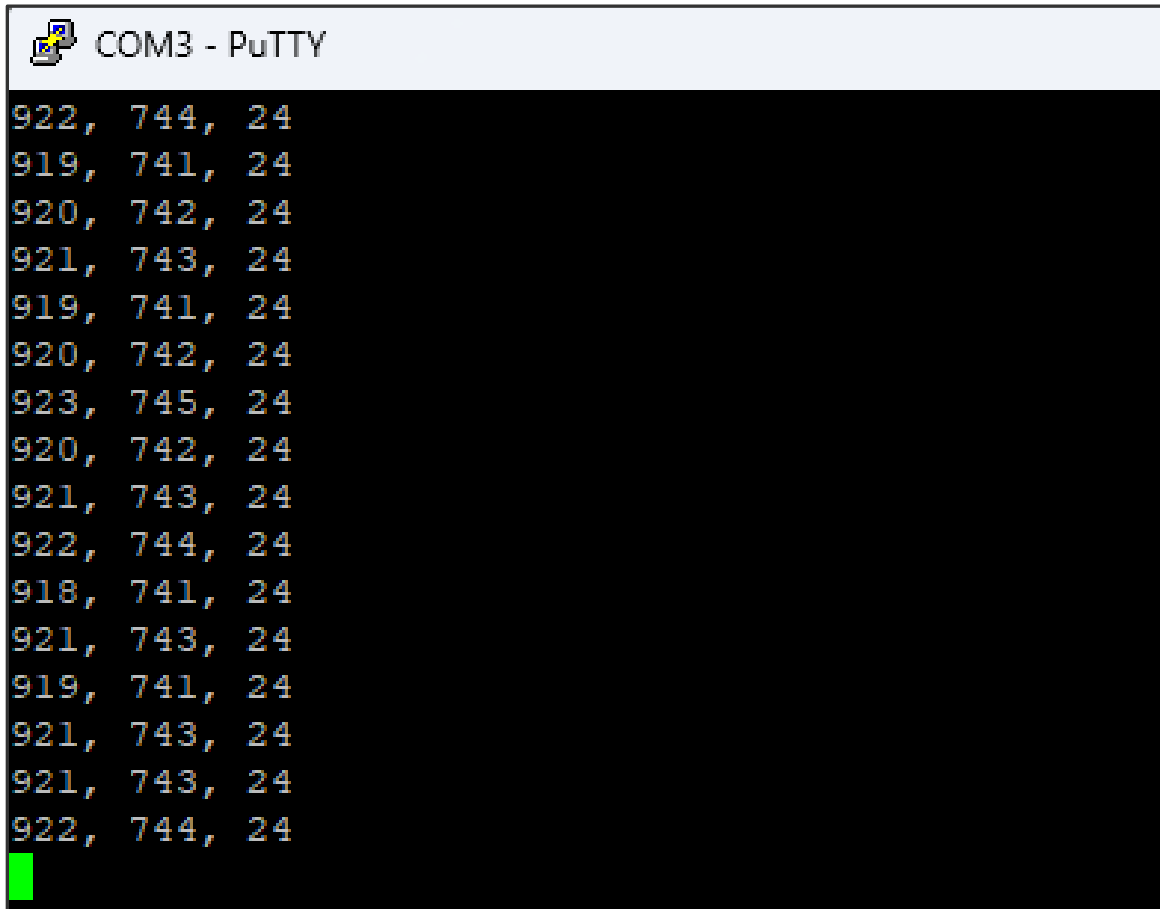


Figure 10: Logic analyzer capture and timing measurements of the PC2 push-button interrupt signal showing negative-logic operation and falling-edge transitions during button actuation.

Figure 11 shows UART temperature sampling data transmitted through the USART2 virtual COM interface following push-button interrupt events. CSV-formatted ADC, voltage, and temperature measurements were successfully displayed within the PuTTY terminal application during repeated measurement trials.



```
COM3 - PuTTY
922, 744, 24
919, 741, 24
920, 742, 24
921, 743, 24
919, 741, 24
920, 742, 24
923, 745, 24
920, 742, 24
921, 743, 24
922, 744, 24
918, 741, 24
921, 743, 24
919, 741, 24
921, 743, 24
921, 743, 24
922, 744, 24
```

Figure 11: UART terminal output showing CSV-formatted ADC, voltage, and temperature data transmitted following push-button interrupt events.

4. Results and Discussion

4.1 ADC and Temperature Response Results

Figure 12 shows the temperature response behavior of the TMP36 sensor under varying thermal conditions. The graph demonstrates gradual temperature recovery following ice-bath exposure, followed by additional increases during elevated-temperature testing. The smooth temperature transitions and stabilization regions confirmed correct ADC acquisition, TMP36 conversion behavior, UART data logging, and MATLAB-based visualization. Minor fluctuations observed throughout the plot were attributed to normal ADC measurement noise and analog sensor variation during continuous sampling. The use of 64-sample ADC averaging significantly reduced measurement fluctuations and improved overall temperature stability during continuous UART-based temperature monitoring.

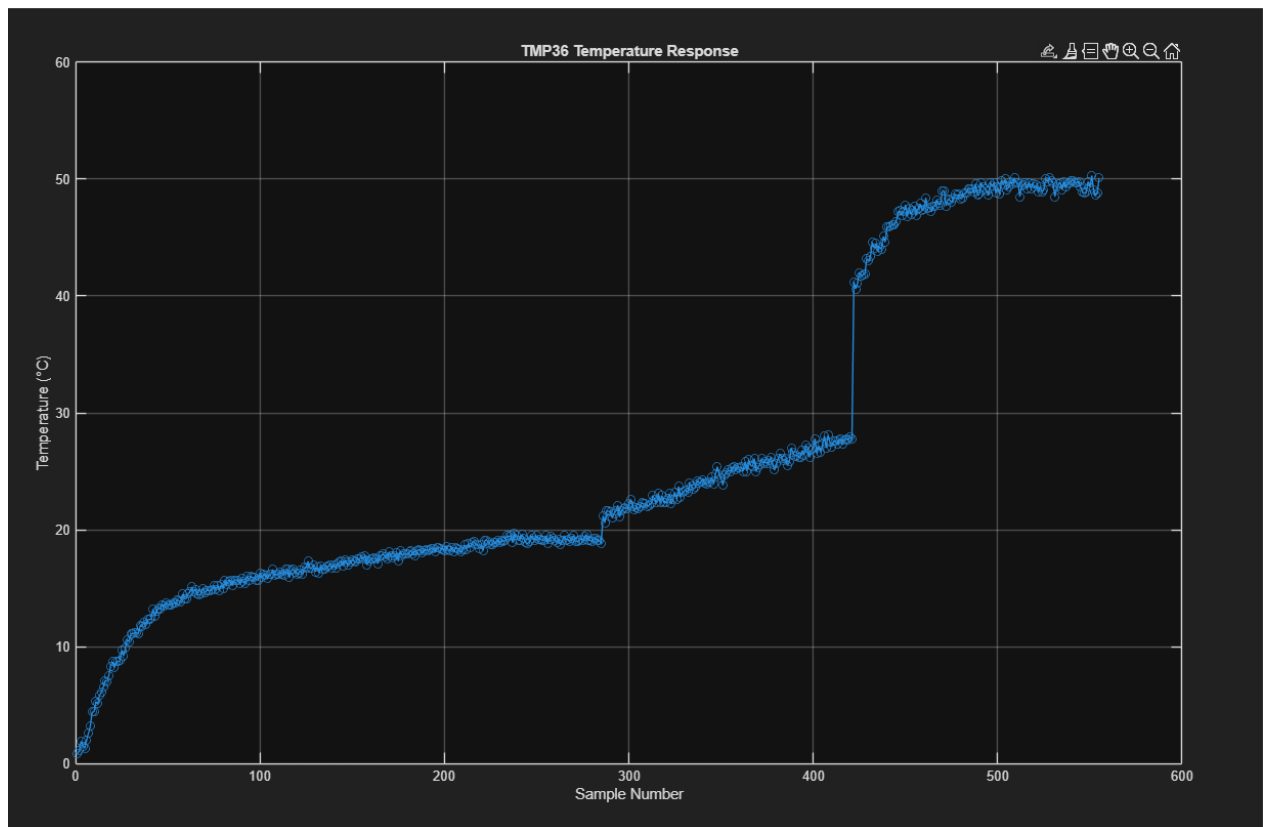


Figure 12: MATLAB temperature response plot generated from CSV-formatted UART sampling data during ice-bath, room-temperature recovery, and elevated-temperature testing of the TMP36 sensor.

The TMP36 sensor exhibited a faster response during elevated-temperature testing compared to room-temperature recovery following ice-bath exposure. This behavior is expected because water provides significantly more efficient heat transfer than ambient air due to its higher thermal conductivity and convective heat transfer capability (Çengel & Ghajar, 2020). The temperature plot also demonstrated gradual thermal stabilization behavior as the TMP36 sensor approached equilibrium within each thermal environment, further confirming realistic sensor response characteristics during dynamic temperature testing.

4.2 UART Communication Results

The UART communication test verified reliable serial transmission of ADC, voltage, and temperature data through the USART2 virtual COM interface. Serial communication between the STM32L476RG microcontroller and the PuTTY terminal application operated successfully throughout all measurement trials.

Figure 13 shows the UART terminal output generated during button-triggered temperature sampling events. CSV-formatted serial data containing ADC counts, measured voltage, and calculated temperature values was transmitted correctly through the USART2 interface at a baud rate of 115,200 bps. The transmitted data remained stable, readable, and consistently formatted during repeated measurement operations.



Figure 13: PuTTY UART terminal output showing CSV-formatted temperature sampling data.

Each push-button interrupt event successfully triggered ADC acquisition and UART data transmission. Multiple measurement trials confirmed that the USART2 peripheral maintained reliable communication without corrupted characters, framing errors, or observable data loss during operation.

The UART communication system also enabled external logging of temperature measurements for subsequent MATLAB analysis and temperature trend visualization. CSV formatting simplified the transfer of recorded data into external analysis tools for further processing and graphical representation.

Overall, the results confirmed stable USART2 serial communication, successful external monitoring of temperature measurements, and reliable integration between ADC acquisition and UART-based data transmission within the interrupt-driven embedded system.

4.3 Push-Button Interrupt and Debounce Results

Figure 14 shows the relationship between the PC2 push-button interrupt waveform, and the corresponding UART temperature output generated by the STM32L476RG system. The logic analyzer waveform demonstrates the negative-logic push-button configuration, where the signal remained at a logic HIGH state while idle and transitioned to logic LOW during button actuation. Each falling-edge transition successfully triggered an external interrupt event, initiating ADC temperature sampling and UART data transmission.

The PuTTY terminal output confirms that valid interrupt events generated CSV-formatted ADC, voltage, and temperature measurements through the USART2 virtual COM interface. Multiple repeated button presses produced stable and consistently formatted UART output, demonstrating reliable interrupt handling and successful SysTick-based debounce operation. Only one temperature transmission event was generated for each valid button press, confirming stable software debounce behavior and preventing unintended multiple interrupt-triggered measurements.

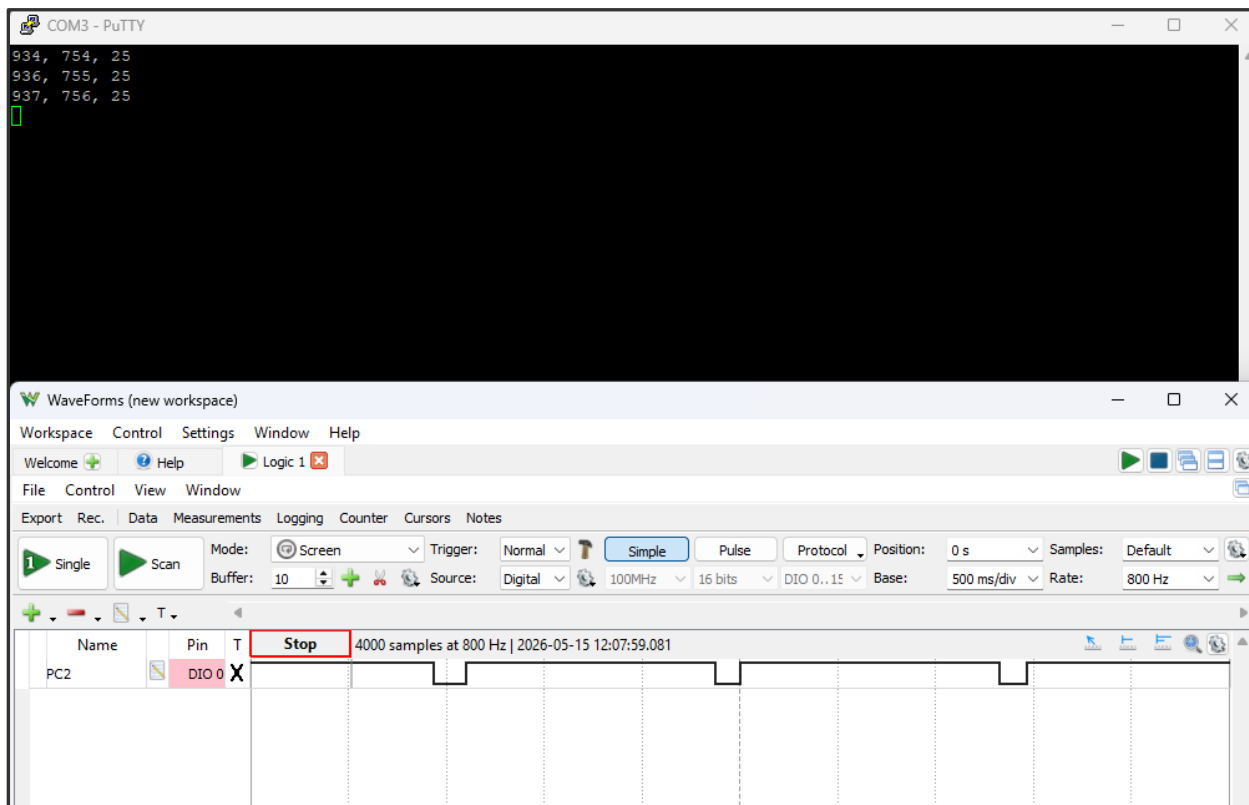


Figure 14: Push-button interrupt waveform and UART temperature data generated during interrupt-triggered ADC sampling.

4.4 Design Constraints

The temperature monitoring system had several design constraints and limitations based on the required hardware and testing approach. The system was implemented using the STM32 Nucleo development board, TMP36 temperature sensor, and push-button input. As a result, overall performance depended on the accuracy of the TMP36 sensor, STM32 ADC resolution, and the reliability of the interrupt-driven push-button interface.

One limitation is that the TMP36 sensor provides an analog voltage output, meaning measurement accuracy depended on ADC conversion resolution, reference voltage stability, and electrical noise within the analog sensing circuit. Although 64-sample ADC averaging significantly improved measurement stability and reduced noise fluctuations, small variations in the measured temperature values were still observed during continuous sampling.

The system also did not use a calibrated reference thermometer, so the collected temperature data was primarily evaluated based on thermal response behavior and trend consistency rather than absolute temperature accuracy. Nevertheless, the measured temperature transitions during ice-bath recovery and elevated-temperature testing demonstrated realistic sensor behavior and appropriate thermal response characteristics.

Another limitation involved the push-button input system, since mechanical switches can produce contact bounce during actuation. Software debounce logic was implemented to minimize repeated triggering and improve reliability, although inconsistent or rapid button presses could still influence sampling operation under certain conditions.

The experimental setup was additionally limited by manually controlled testing conditions. Temperature variation was generated using an ice bath, room-temperature exposure, and warm-water testing, which provided practical dynamic temperature changes but did not offer tightly controlled environmental conditions or precision thermal calibration.

Overall, the system successfully demonstrated event-driven temperature measurement, ADC acquisition, UART data transmission, CSV-based data logging, and MATLAB visualization. While measurement precision and repeatability were limited by sensor tolerance, ADC noise, switch bounce, and manual testing conditions, the system produced stable and realistic temperature response behavior throughout dynamic thermal testing.

4.5 Economic Constraint

The temperature monitoring system was designed using low-cost and widely available embedded system components, including the STM32L476RG Nucleo development board and TMP36 analog temperature sensor. The overall hardware requirements were minimal and did not require dedicated external communication hardware, specialized sensing equipment, or additional display interfaces.

UART communication through the onboard virtual COM port reduced system complexity and eliminated the need for external serial adapters or standalone monitoring hardware. The use of reusable laboratory equipment and standard breadboard-based prototyping components also minimized overall implementation cost.

Overall, the project demonstrated that an interrupt-driven embedded temperature monitoring system could be implemented using inexpensive and accessible hardware while maintaining reliable sensing, interrupt handling, ADC acquisition, and UART communication functionality.

4.6 Sustainability Constraint

The implemented temperature monitoring system incorporated several sustainability-related considerations through low-power operation, reusable hardware utilization, and minimal component requirements. Event-driven temperature sampling using push-button interrupts reduced unnecessary continuous ADC sampling and UART communication activity, lowering overall system processing overhead and power consumption.

The system utilized a minimal hardware configuration consisting primarily of the STM32L476RG Nucleo development board, TMP36 temperature sensor, and basic peripheral interfaces. Reusable laboratory instrumentation, including the Analog Discovery 2 (AD2) logic analyzer and virtual COM port interface, reduced the need for additional dedicated measurement hardware.

UART-based digital data transmission and MATLAB-based visualization supported electronic data collection and analysis without requiring standalone display hardware or printed measurement records.

Overall, the project demonstrated how interrupt-driven embedded system architectures can support efficient operation while minimizing hardware complexity and resource usage.

4.7 Environmental Constraint

The implemented temperature monitoring system was evaluated under varying environmental temperature conditions using an ice bath, room-temperature ambient conditions, and warm-water exposure of the TMP36 sensor. These conditions were used to observe the response behavior of the ADC subsystem and software-based temperature conversion algorithm under changing thermal environments.

During testing, the TMP36 sensor produced corresponding voltage and temperature changes as environmental conditions varied. Lower measured temperatures were observed during ice-bath exposure, while elevated temperature values were produced during warm-water exposure. The system maintained stable UART communication and ADC acquisition behavior throughout all testing conditions.

The project demonstrated that the STM32L476RG ADC subsystem and TMP36 sensor interface could operate reliably across varying ambient temperature conditions commonly encountered in basic environmental sensing applications.

5. Roles of Team Members

Avery Whann was responsible for embedded software development, including ADC configuration, TMP36 sensor integration, EXTI interrupt handling, SysTick timing, UART communication, and overall system integration. Avery was also responsible for report writing, report formatting, experimental documentation, and MATLAB data analysis.

Yosias Samuel was responsible for developing the presentation slides, writing the resume-ready project summary, and creating key project figures including the software flowchart, circuit diagram, and system block diagram. Yosias also contributed to project organization, report writing, documentation review, and presentation preparation.

Shahroz Rizwan was responsible for presentation preparation and project demonstration support. Shahroz also assisted with hardware setup, circuit design, component selection, project selection, testing, and overall project coordination.

6. Conclusion and Future Work

This project successfully implemented an interrupt-driven temperature monitoring system using the STM32L476RG microcontroller and TMP36 analog temperature sensor. The system integrated ADC acquisition, EXTI interrupt handling, SysTick timing, and USART2 serial communication into a modular embedded software architecture capable of user-triggered temperature measurement and external data logging.

The project demonstrated how analog sensor data can be acquired, converted into digital measurements, processed through software-based temperature conversion, and transmitted through UART for external monitoring and analysis. The interrupt-driven design allowed temperature sampling to occur only when requested through push-button input, supporting event-driven embedded system operation.

The modular software structure also provided practical experience with peripheral configuration, interrupt handling, ADC calibration, debounce timing, and serial communication within an STM32 embedded environment.

Future improvements for the system could include implementation of low-power sleep modes, wireless data transmission, LCD or OLED display integration, continuous background temperature logging, and higher-precision sensor calibration. Additional improvements could include development of a more advanced debounce system using hardware filtering or timer-based debounce logic to further improve push-button input handling.

One limitation of the prototype hardware design was the absence of capacitors near the STM32L476RG and TMP36 power connections. While the breadboard implementation operated successfully for demonstration purposes, future hardware revisions should include appropriate capacitors to improve power supply stability, reduce electrical noise, and improve ADC measurement accuracy. Additional hardware improvements could also include dedicated reset and boot circuitry, improved PCB grounding, and integrated programming/debugging headers for a more robust embedded hardware design.

Future versions of the project could also transition from a breadboard prototype to a permanent PCB-based design housed inside a custom 3D-printed enclosure. Integrating a dedicated battery power supply would further support the development of a compact portable temperature monitoring device intended for long-term standalone operation.

Overall, the system successfully demonstrated reliable interrupt-driven sensing, stable UART communication, and accurate temperature trend monitoring using low-cost embedded hardware.

7. References

References

Analog Devices. (n.d.). *TMP35/TMP36/TMP37 Low Voltage Temperature Sensors Data Sheet*.

https://www.analog.com/media/en/technical-documentation/data-sheets/TMP35_36_37.pdf

Çengel, Y. A., & Ghajar, A. J. (2020). *Heat and mass transfer: Fundamentals and applications* (6th ed.).

McGraw-Hill Education.

<https://www.mheducation.com/highered/product/heat-and-mass-transfer-fundamentals-and-applications-cengel.html?viewOption=student>

STMicroelectronics. (2015). *STM32 Nucleo-64 boards data brief (DocID025838 Rev 7)*.

https://www.st.com/resource/en/data_brief/nucleo-f207zg.pdf

STMicroelectronics. (2020). *RM0351 reference manual: STM32L47xxx, STM32L48xxx, STM32L49xxx and STM32L4Axxx advanced Arm-based 32-bit MCUs*.

https://www.st.com/resource/en/reference_manual/rm0351-stm32l47xxx-stm32l48xxx-stm32l49xxx-and-stm32l4axxx-advanced-armbased-32bit-mcus-stmicroelectronics.pdf

8. AI Use Statement

We used ChatGPT to refine phrasing, aid with formatting lists and improve document structure, and to identify and correct minor grammar and spelling errors. We reviewed and revised all content ourselves to ensure technical accuracy and to demonstrate our own understanding. All code implementation, debugging, and analysis were completed independently.